COSC 4370 – Homework 4

Name: Samuel Heng PSID: 1580496

September 2023

1 Problem

This assignment requires the implementation of texture mapping in OpenGL to reproduce a rotating textured cube. The goal is to effectively demonstrate the practical usage of UV mapping, texture rendering, and the utilization of shaders in OpenGL.

2 Method

There are four files that need modification in the provided codes: *Camera.h, main.cpp, texture.frag,* and *texture.vs.* In order of least modification to most: *Camera.h* implements the camera movement, orientation, and view matrix; *main.cpp* loads a DDS texture used to render the desired cube; *texture.vs* produces adjusted UV coordinates for texture mapping; and *texture.frag* which applies the sample texture to the fragment at the given coordinates.

The general idea behind this method is to load the DDS texture and set up a UV buffer to transfer the vertex, and UV coordinates to the vertex shader. The vertex shader computes and passes the adjusted data to the fragment shader to correctly place the corresponding textures on each fragment on the cube's surfaces.

3 Implementation

Given the provided files to modify, *main.cpp, Camera.h, texture.frag, texture.vs*; the implementation involves setting the environment, loading the textures, and rendering the transformed and textured 3D rotating cube.

3.1 main.cpp

The main purpose of this file is the rendering of a rotating textured cube in a 3D viewing window. This includes defining the cube's geometry by setting up the vertices and UV coordinates, and creating a *Vertex Array Object (VAO),* and a *Vertex Buffer Object*

(VBO) to store the data. Next the view and projection matrices are passed to the vertex shader for further processing.

Many steps are taken before the final render is produced. Firstly, to achieve the textured cube, we must load the DDS texture file using:

$$GLuint\ texture = loadDDS(\text{"texture.dds"});$$

The texture is then bound to a texture object which is used to produce the desired result.

Next, the UV buffer is set up to map the textures to the 3D objects:

$$glGenBuffers(1, \&UVBO);$$
$$glBindBuffer(GL_ARRAY_BUFFER, UVBO);$$
$$glBufferData(GL_ARRAY_BUFFER, sizeof(uv), uv, GL_STATIC_DRAW);$$

The function *glGenBuffers();* generates the buffer object and returns the object's ID to use in *glBindBuffer()* where the object is bound to target vertex attributes and UV coordinates where the data is then sent using *glBufferData()*. After setting up the buffer, the UV attributes must be configured:

$$glVertexAttribPointer(1,2,G_FLOAT, GL_FALSE, 0, (GLvoid*)0;$$
$$glEnableVertexAttribArray(1);$$

The function *glVertexAttribPointer()* is responsible for specifying the attributes of the UV coordinates in the buffer. Finally, *glEnableVertexAttribArray()* makes the attribute accessible to the vertex shader.

Secondly, the projection matrix is set up using:

$$glm::perspective(glm::radians(45.0f), (GLfloat)WIDTH/HEIGHT, 0.1f, 100.0f);$$

glm::perspective is a function that is used to create the projection matrix by converting vectors from eye space to clip space. This is important to simulate a realistic depth and perspective in respect to the height and width of the viewing window.

Last and most important, to apply the texture to the render, the loaded texture must be bound to the object using:

```
glActiveTexture(GL_TEXTURE0);
```

```
glBindTexture(GL_TEXTURE_2D, texture);
glUniform1i(textureID, 0);
```

The first two functions: *glActiveTexture()* and *glBindTexture()* work in unison to bind the loaded texture onto a given texture object. The last function, *glUniform1i(),* sets up the uniform, *myTextureSampler,* to the corresponding texture object in the vertex shader. All three of these functions work together to ensure that the shader is sampling from the correct texture object.

3.2 Camera.h

The camera class is an essential component to this assignment because it handles the camera's position and direction. The *GetViewMatrix()* function is responsible for generating and returning the view matrix. This is implemented by calling the GLM function:

$$glm::lookAt(Position, Position + Front, Up);$$

glm::lookAt() creates the view matrix using: '*Position*' which is the camera's position in the world space, '*Position + Front*' which defines the camera's viewing direction, and '*Up*' which is the upward direction vector that determines the camera's orientation and rotation point. Overall, the view matrix allows for the transformation of objects from world space to camera space. This file specifically handles user keyboard and mouse input to control movement and orientation, enabling viewers to easily navigate the camera window.

3.3 texture.vs

Another essential component of the rendering pipeline is the vertex shader. In this case, the shader transforms the given vertex positions in respect to the model, view, and projection matrix, and adjusts the UV coordinates which are passed to the fragment shader for texture sampling.

The implementation starts with the transformation of the vertex position using :

$$gl_Position = projection * view * model * vec4(position, 1.0);$$

Here we are transforming the local coordinates of the model to bring the vertex into the 2D viewing space by applying the perspective projection.

Most importantly, the UV coordinates must be adjusted to ensure that the texture is applied correctly and uniformly.

$$UV = vec2(vertexUV.x, 1.0 - vertexUV.y);$$

The variable *vertexUV* contains the UV coordinates for the given vertex, and *UV* represents the adjusted UV coordinates passed to the fragment shader for processing. The y coordinates of the UV must be inverted to match the texture coordinates, otherwise the textures will either render flipped, or not at all. This ensures that each texture is uniformly applied on each corresponding surface.

3.4 texture.frag

To achieve the completed rendering, the fragment shader takes the interpolated UV coordinates from the vertex shader, samples the 2D texture, and applies this texture to the corresponding face on the cube's surface.

The fragment shader creates the variable, *in vec2 UV,* to store the coordinates passed by the vertex shader. This specifies the position of the current fragment. Next we have the uniform variable, *myTextureSampler* which is used to access the specific bound 2D texture object.

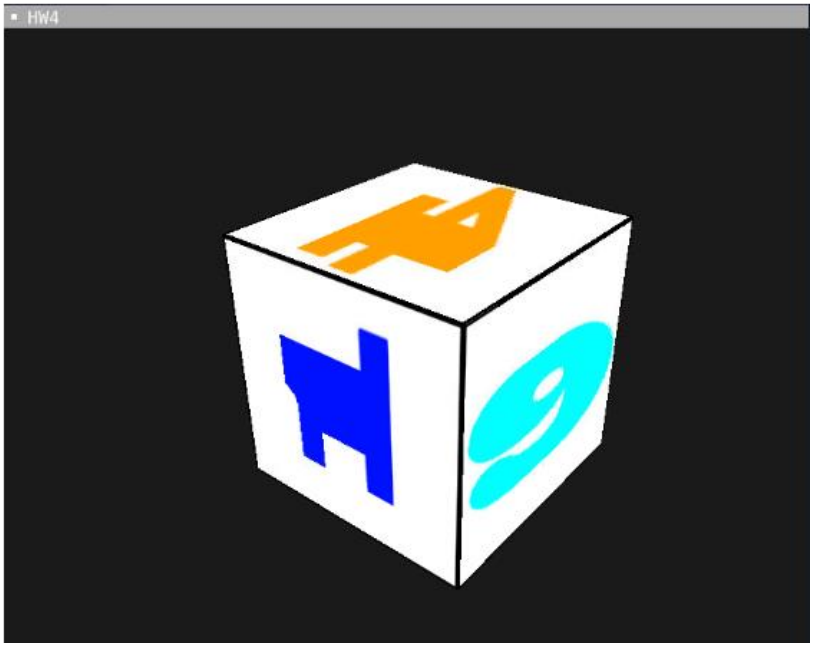
For each fragment of the cube's surface:

$$color = texture(myTextureSampler, UV);$$

The function *texture()* samples the 2D texture from *myTextureSampler* at the given UV coordinates to retrieve and assigns the texture at the corresponding fragment position. This is a fundamental step in rendering realistic textures on 3D models.

4 Result

The resulting output is a viewing window featuring an angled topdown view of a textured 3D cube. The cube is rotating counterclockwise, and each surface of the cube is textured with a uniquely colored number.



Sources used:

- Textures by LearnOpenGL(provided in HWK pdf)